DOCUMENTATION TECHNIQUE · 12 NOVEMBRE 2026

Architecture de l’application WEB

Outil interne B2B multi-sociétés — groupe de quincailleries de Nouvelle-Calédonie

Backend Node.js / Express · Frontend React (CRA) · MongoDB · fichiers DBF de l’ERP

ORDRES DE GRANDEUR

Couche	Volume
Backend	~106 600 lignes — **88 routeurs**, **90 contrôleurs**, **106 services**, **67 modèles** (+5 masterConfig)
Frontend	~87 900 lignes — **120 écrans**, **81 slices** RTK Query / Redux, ~50 composants
App collecteur	~16 800 lignes (dépôt git séparé, `test-app-qc/QcApp`)
Tests	**aucun** — ni back, ni front, ni mobile

COMMENT LIRE CE DOCUMENT

Une page A3 par diagramme. L’entête rappelle la section, le pied de page la pagination. Les encadrés bleus sous un diagramme portent les précisions et les arbitrages clients associés.

Établi par lecture du dépôt, sans supposition : routeurs, contrôleurs, services, modèles, écrans et slices ont été énumérés sur le code.

LÉGENDE DES COULEURS

- Interdit, piège grave ou dette ouverte
- Point de vigilance à vérifier
- Source de vérité · règle à suivre
- Donnée en LECTURE SEULE (ERP)

Sommaire

1 Vue d'ensemble de la plateforme 3

2 Les deux sources de données — le cœur de l'architecture 4

Les trois familles de cache DBF 5
Règles DBF non négociables 6

3 Chaîne de traitement d'une requête API 7

4 Authentification, rôles et permissions 8

Gardes complémentaires et droits fins 9
Le registre des modules vit à TROIS endroits 10

5 Frontend — pile et gardiennage des routes 11

6 Carte des domaines fonctionnels — 88 routeurs 12

Domaines — Terrain 13
Domaines — Données et Analyse 14
Domaines — Communication, Administration, API partenaire 15
Deux « suivis bipage » à ne pas confondre 16
Réappro et bipage ne posent PAS la même question 17

7 Modèle de données MongoDB 18

Modèle de données — utilisateurs, équipes et échanges 19
Modèle de données — les flux du terrain 20
Modèle de données — mailing et configuration 21
Les champs qui portent les décisions 22

8 Tâches de fond et chaîne d'impression 23

Impression des fiches de contrôle — qui fait quoi 24
Destination du dossier d'inventaire — on stocke le MODE, jamais un chemin 25

9 Flux inventaire par zones — de bout en bout 26

Du dépôt à la fiche imprimée 27

10 Espace commercial — découpage imposé par la perf 28

Deux sources distinctes, ne pas les confondre 29

11 API partenaire — /api/public/v1 30

12 Temps réel — Socket.IO 31

13 Bascule dev/prod et variables d'environnement 32

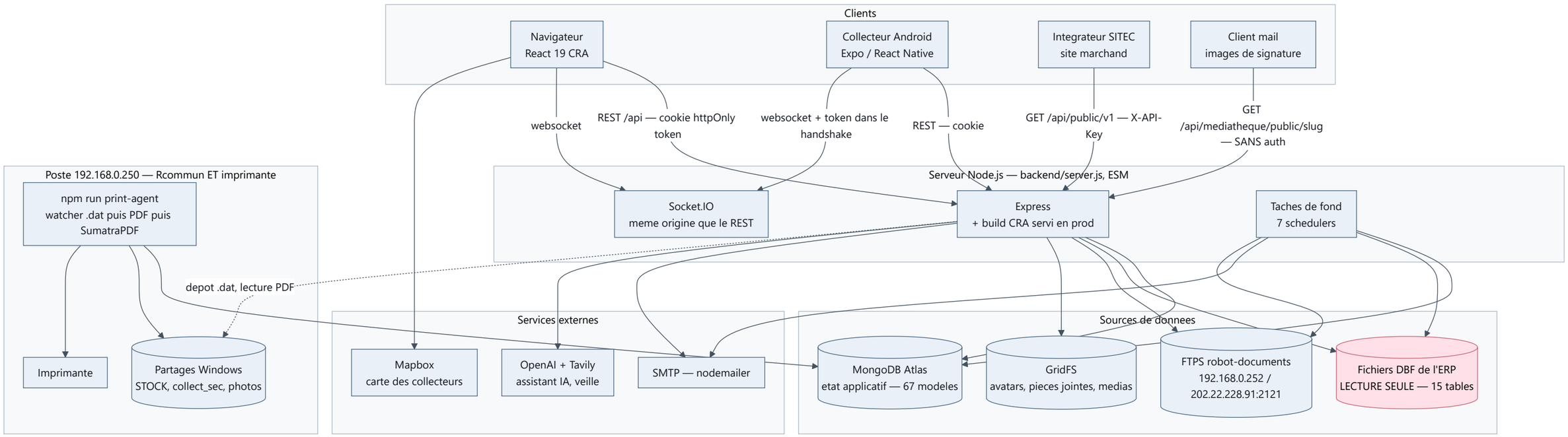
Commandes 33

14 Pièges connus et dette identifiée 34

Deux modules jumeaux, deux règles métier OPPOSÉES 35

- **Conventions de code — rappel** 36

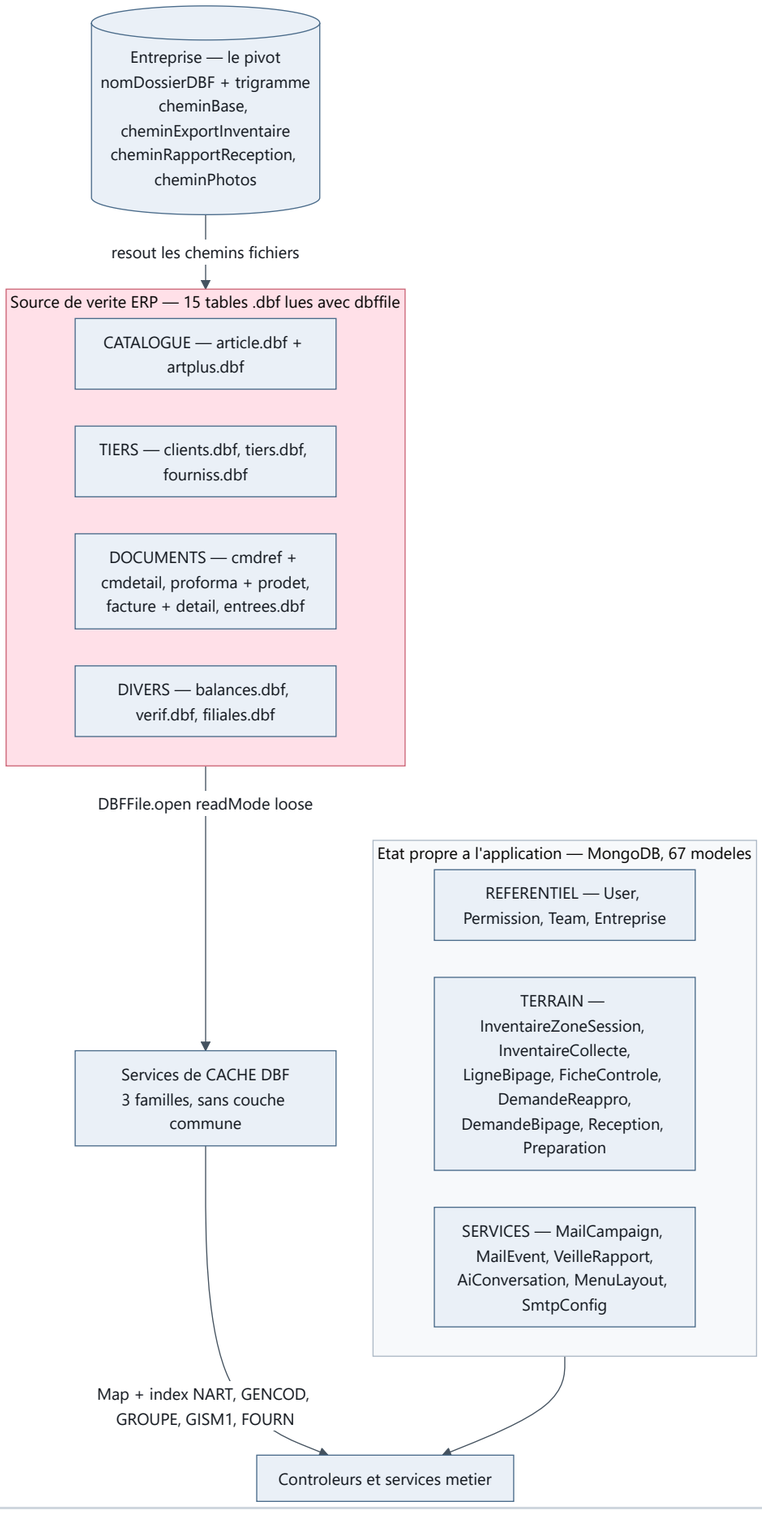
Qui parle à qui, et par quoi.



À retenir

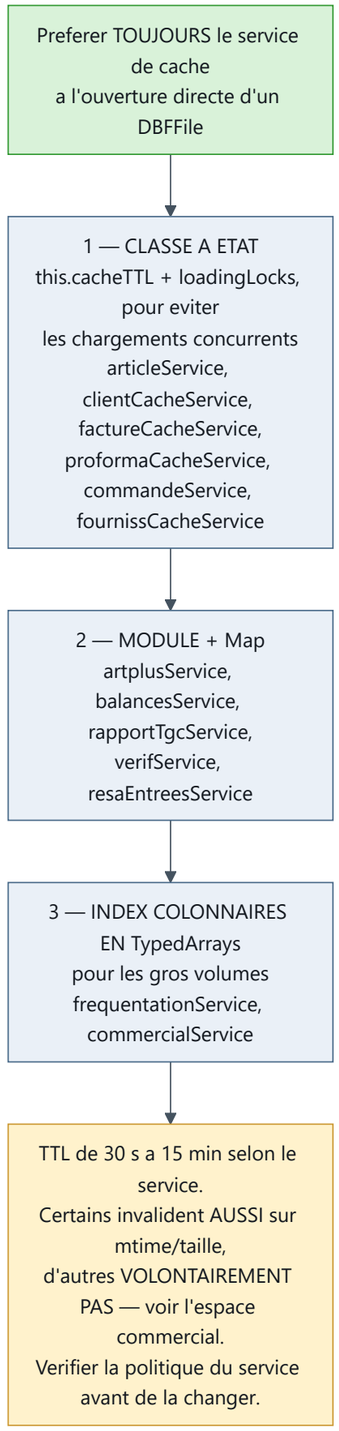
- L'ERP est **strictement en lecture** : aucune écriture DBF n'existe dans le code, et il ne doit pas en être ajouté.
- Le backend de production est un **VPS Ubuntu qui ne peut pas imprimer** ; l'impression des fiches de contrôle est portée par l'agent local sur `192.168.0.250`, branché au **même** Mongo Atlas.
- `BASE_URL = ""` côté front : **même origine** — proxy CRA en dev, Express servant le build en prod.

Ce n'est **pas** une application CRUD Mongo. Deux backends cohabitent, et tout ce qui vient du DBF est scopé par société via le paramètre de route `:nomDossierDBF`.

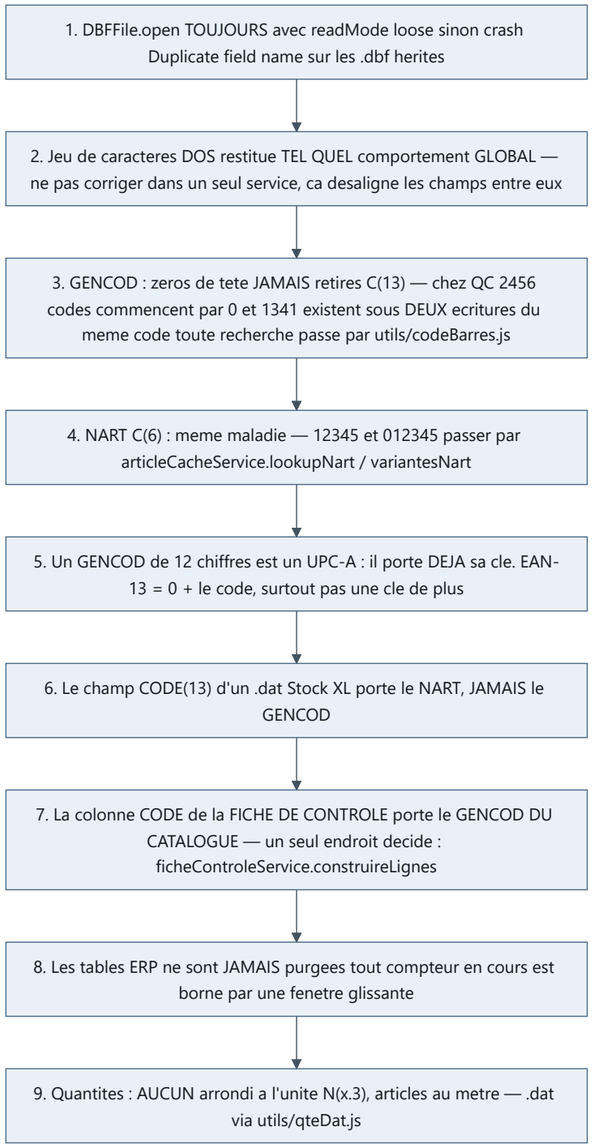


Les trois familles de cache DBF

Relire le DBF à chaque requête est trop lent — 90 000+ articles, **1,7 M factures et 6,2 M lignes** chez QC.

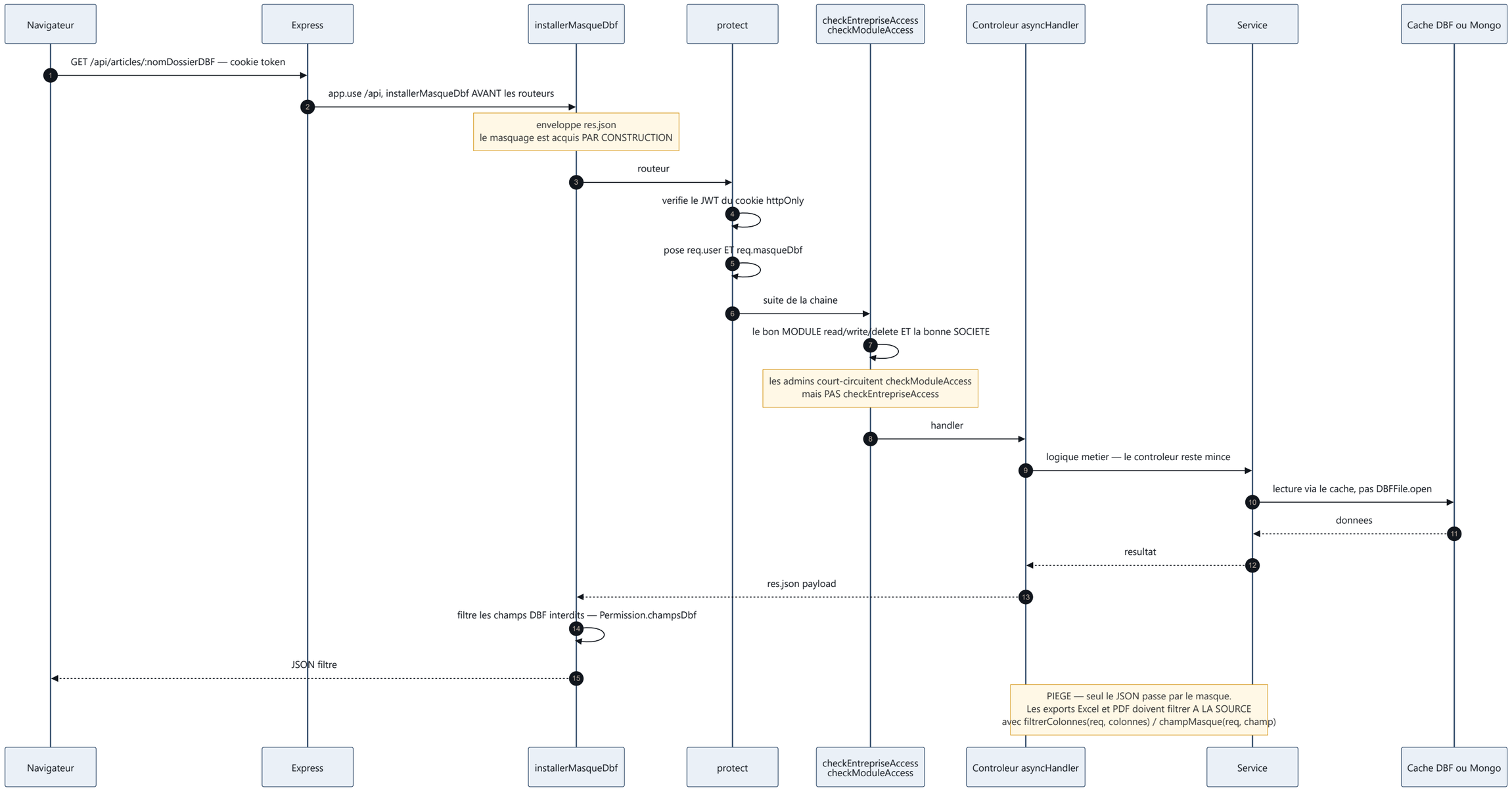


Règles DBF non négociables

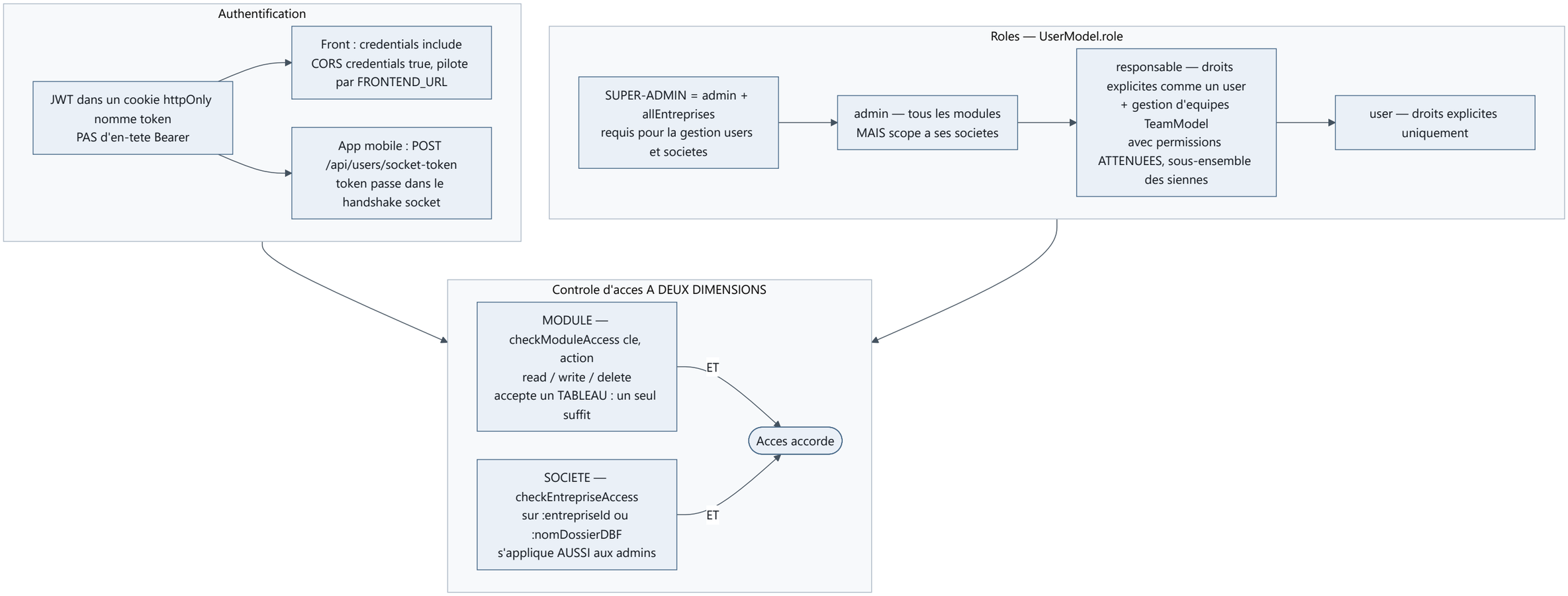


Trois codes à ne pas confondre sur la chaîne d'inventaire :

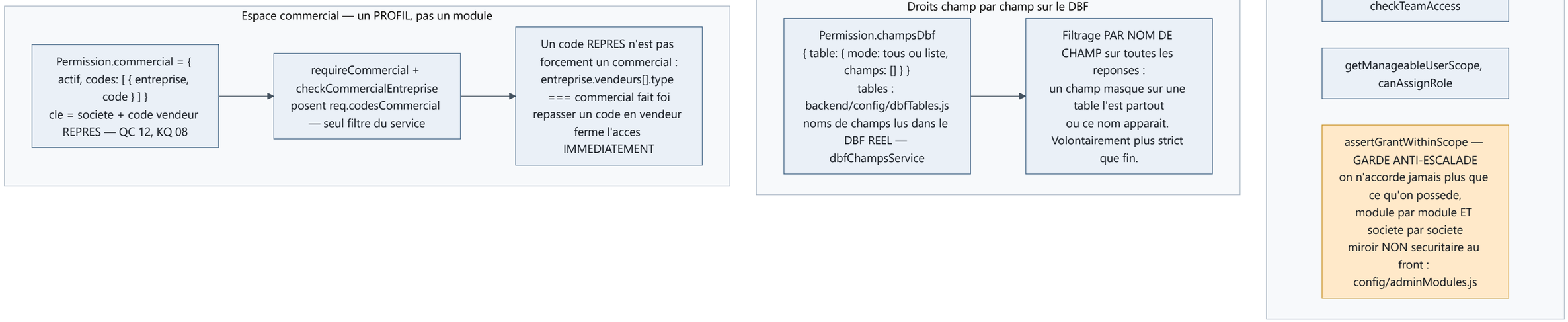
Usage	Code employé	Décidé dans
Écriture du `.dat` Stock XL	**NART**	`genererContenuFichier`
Résolution de l'article	**code réellement bipé**	`InventaireCollecte.lignes[].codeScanne`
Affichage fiche de contrôle	**GENCOD du catalogue**	`ficheControleService.construireLignes`



> Les erreurs remontent à `notFound` puis `errorHandler` (`backend/middleware/errorMiddleware.js`). Un nouveau routeur doit être **importé ET monté** (`app.use()`) dans `backend/server.js` .


Authentification
JWT dans un cookie httpOnly
nomme token
PAS d'en-tete Bearer
Front : credentials include
CORS credentials true, pilote
par FRONTEND_URL
App mobile : POST
/api/users/socket-token
token passe dans le
handshake socket
Roles — UserModel.role
SUPER-ADMIN = admin +
allEntreprises
requis pour la gestion users
et societes
admin — tous les modules
MAIS scope a ses societes
responsable — droits
explicites comme un user
+ gestion d'equipes
TeamModel
avec permissions
ATTENUEES, sous-ensemble
des siennes
user — droits explicites
uniquement
Controle d'acces A DEUX DIMENSIONS
MODULE —
checkModuleAccess cle,
action
read / write / delete
accepte un TABLEAU : un seul
suffit
ET
Acces accorde
SOCIETE —
checkEntrepriseAccess
sur :entrepriseId ou
:nomDossierDBF
s'applique AUSSI aux admins
ET

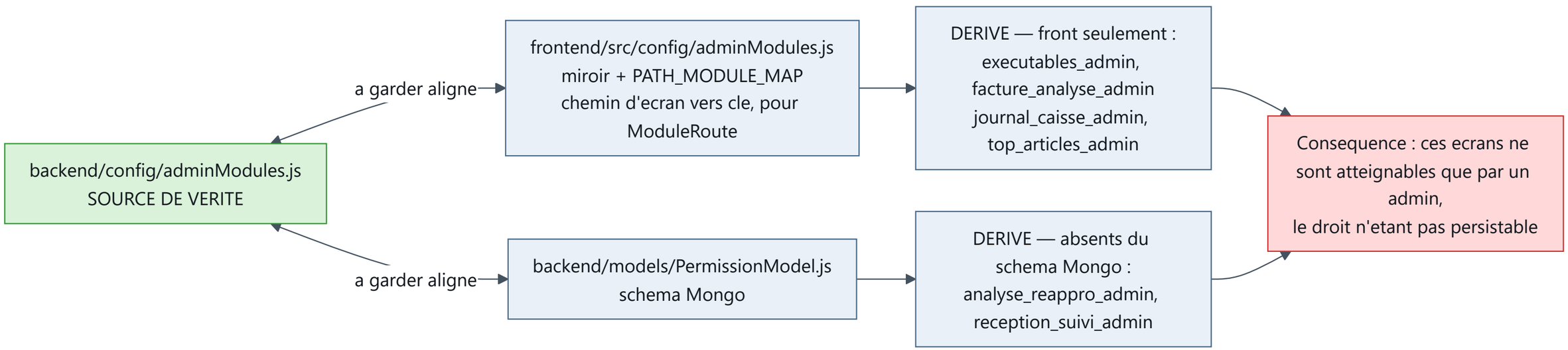
Gardes complémentaires et droits fins



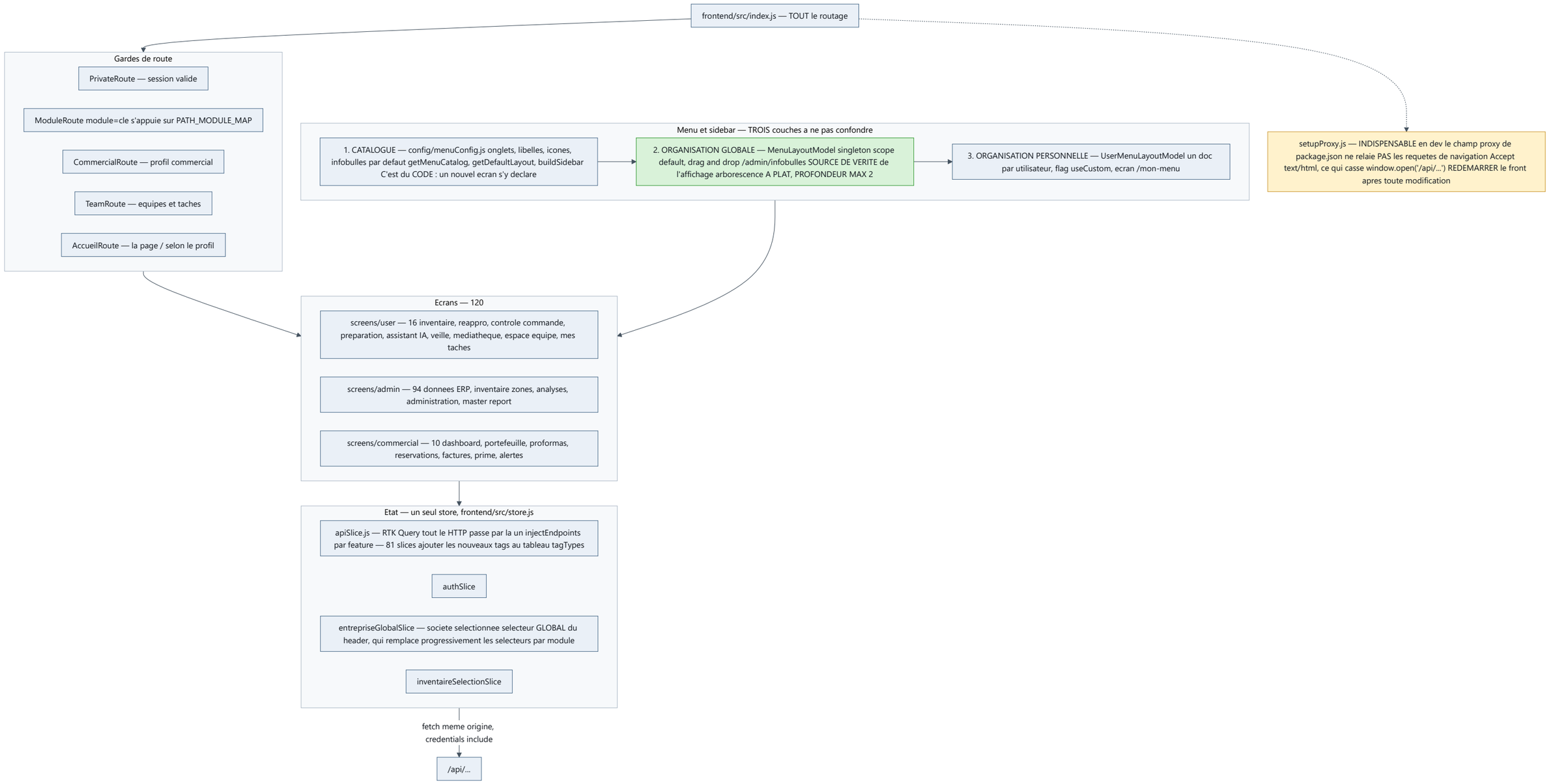
Exception assumée au scoping société : `GET /api/inventaires-zones/:entrepriseId/agents-possibles` renvoie **tous** les comptes actifs — un inventaire est régulièrement renforcé par du personnel d'une autre société du groupe, qui doit pouvoir être crédité de son travail. La route reste `admin` + `checkEntrepriseAccess` et ne renvoie que l'identité (nom, e-mail).

Décision client : un responsable gère **tous** les utilisateurs de ses sociétés, pas seulement les membres de son équipe.

Le registre des modules vit à TROIS endroits

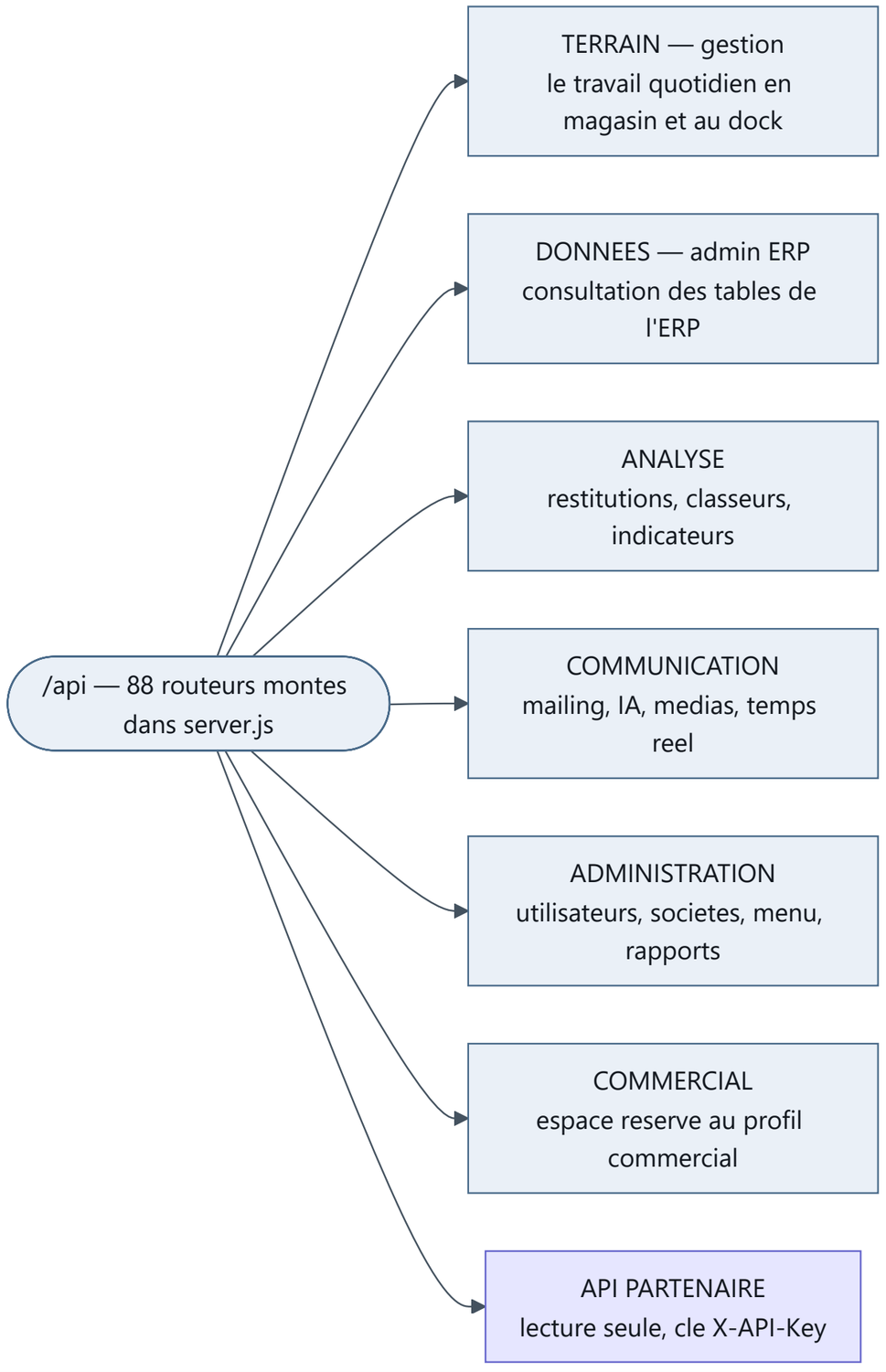


> **En ajoutant un module, mettre à jour les trois.** Vérifier ces trois fichiers avant de conclure qu'un droit « ne marche pas ».

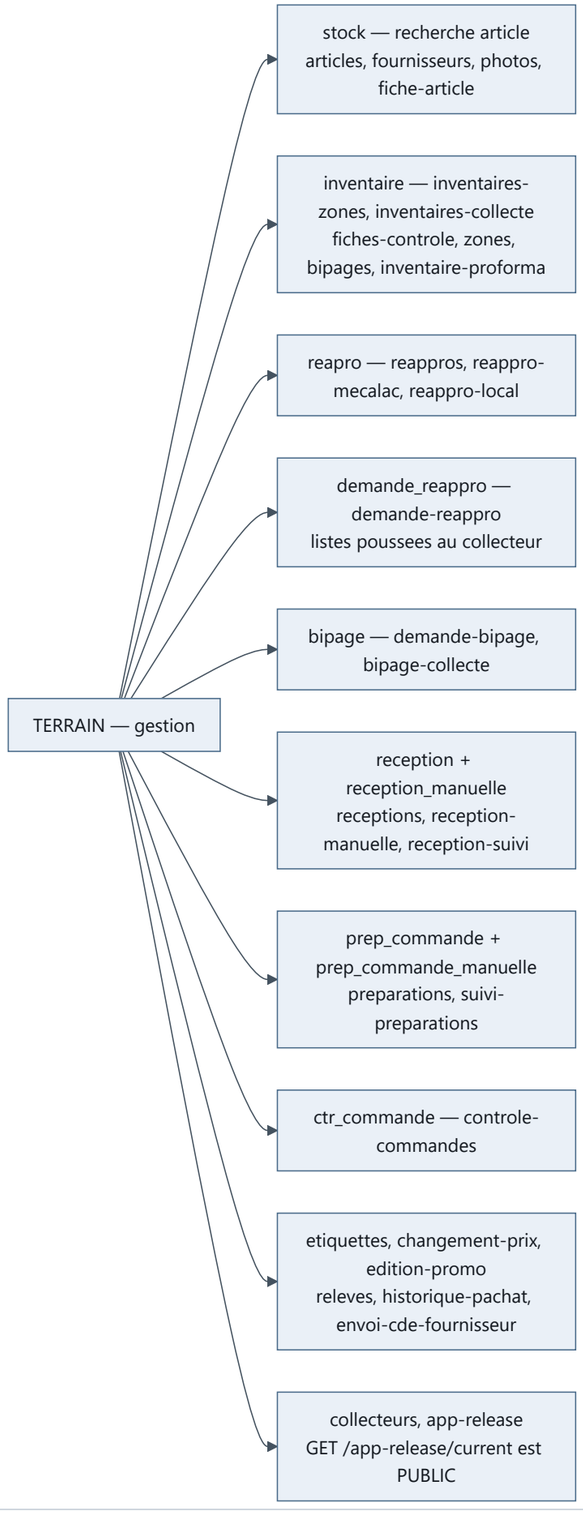


> **Sidebar** : elle s'ouvre **entièrement repliée** (décision client — l'état des dossiers n'est plus persisté en localStorage). Le champ de recherche sous le switch Défaut/Perso est le chemin rapide : il aplatit tout le menu, filtre sans accent ni casse, et affiche le chemin du dossier sous chaque résultat.
>
> Si un lien de sidebar « manque » alors qu'il est bien dans le DOM : suspecter `max-height` / `overflow` **avant** les permissions.

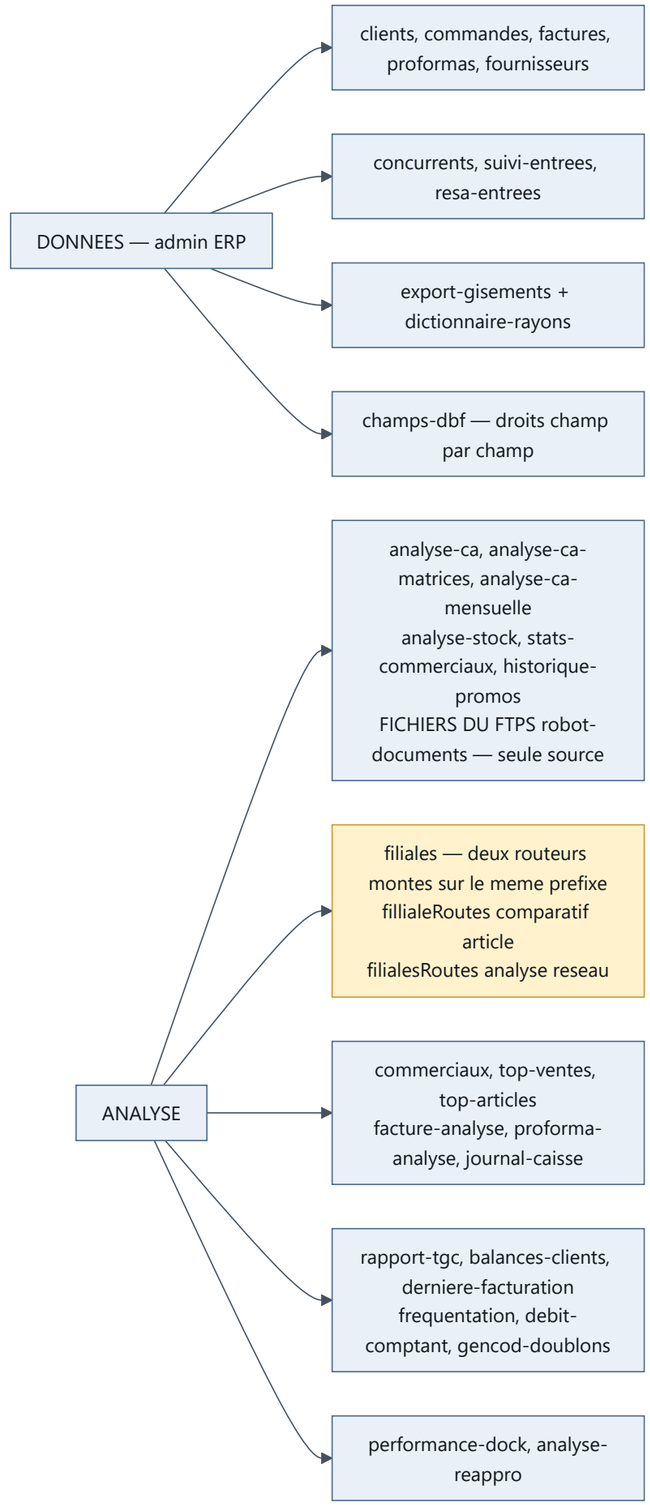
Repères pour situer un écran ou un routeur. Les préfixes API sont sous `/api/`.



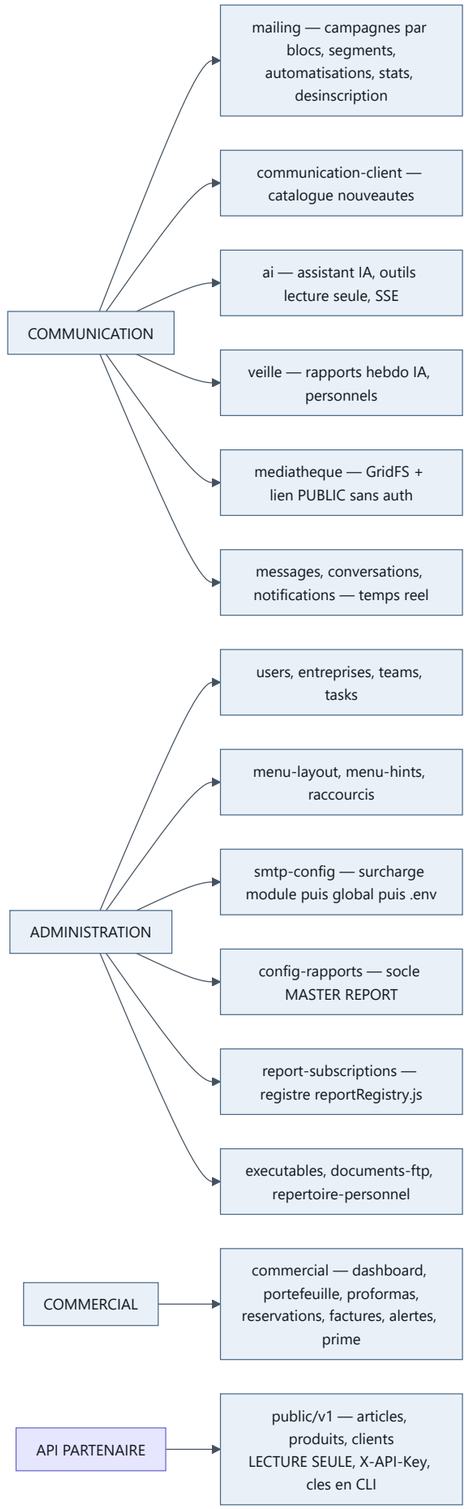
Domaines — Terrain



Domaines — Données et Analyse



Domaines — Communication, Administration, API partenaire



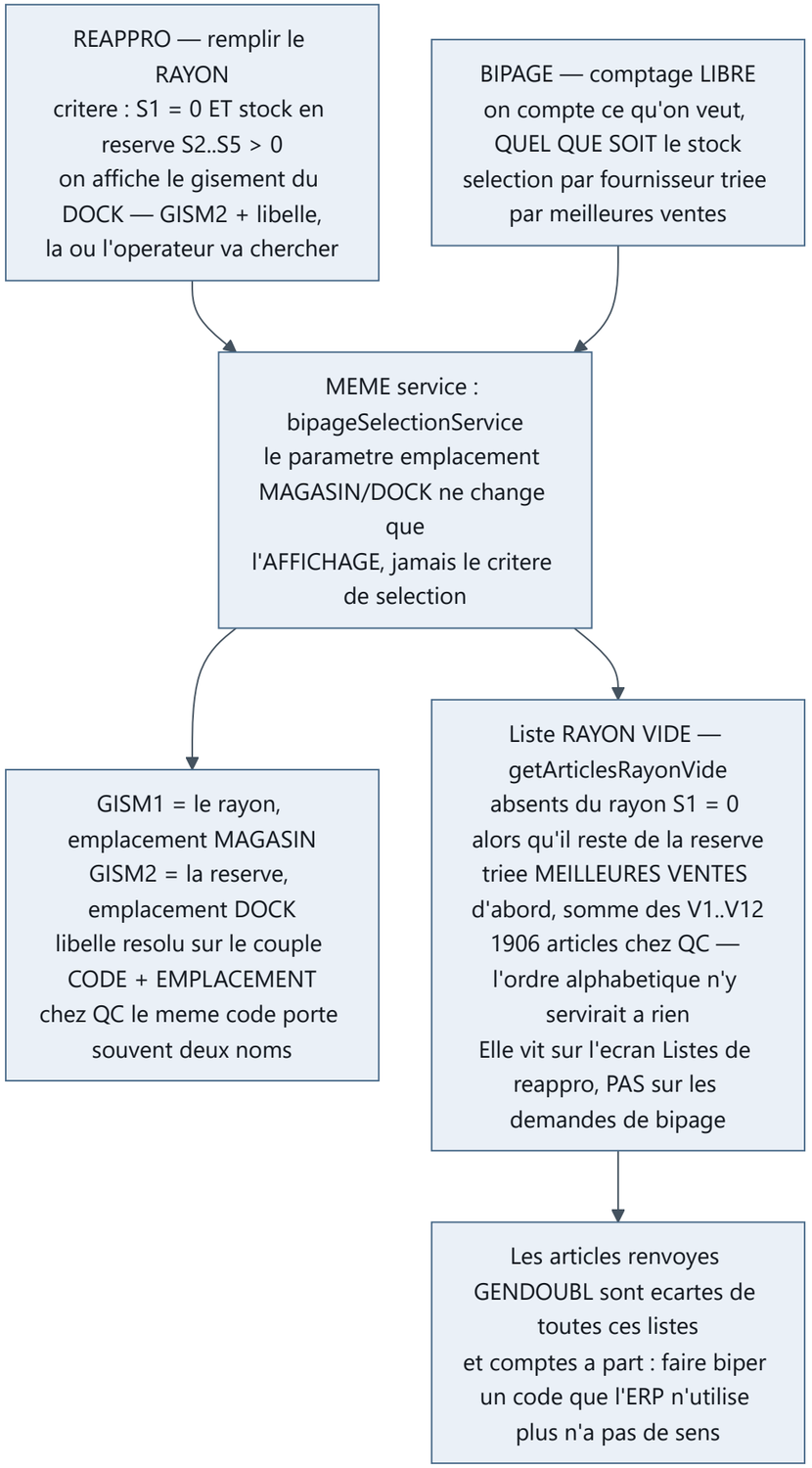
Deux « suivis bipage » à ne pas confondre

/admin/suivi-demandes-bipage — module bipage
suit les DEMANDES envoyees aux collecteurs depuis le web
qui a bipe quoi, combien de lignes, en combien de temps
vit avec les documents DemandeBipage
Dossier : Terrain puis Bipage

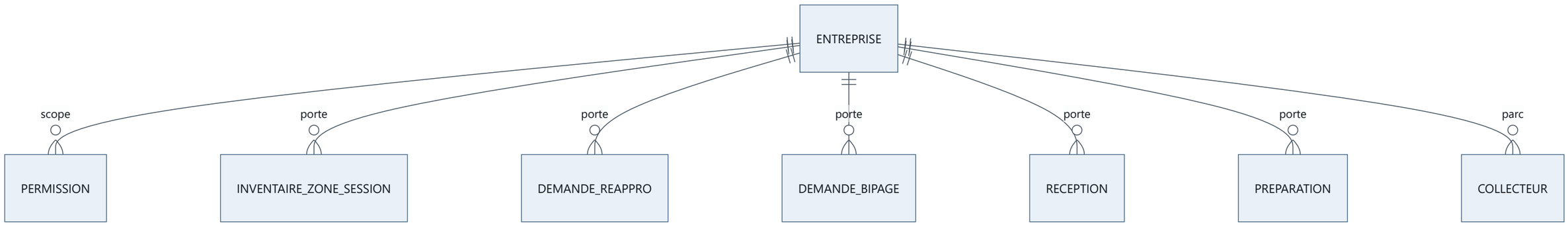
Memes mots, deux ecrans, deux modules

/admin/suivi-bipage — module inventaire
suit le COMPTAGE D'INVENTAIRE par zone
se vide a chaque reinitialisation d'inventaire
Dossier : Inventaire Zones

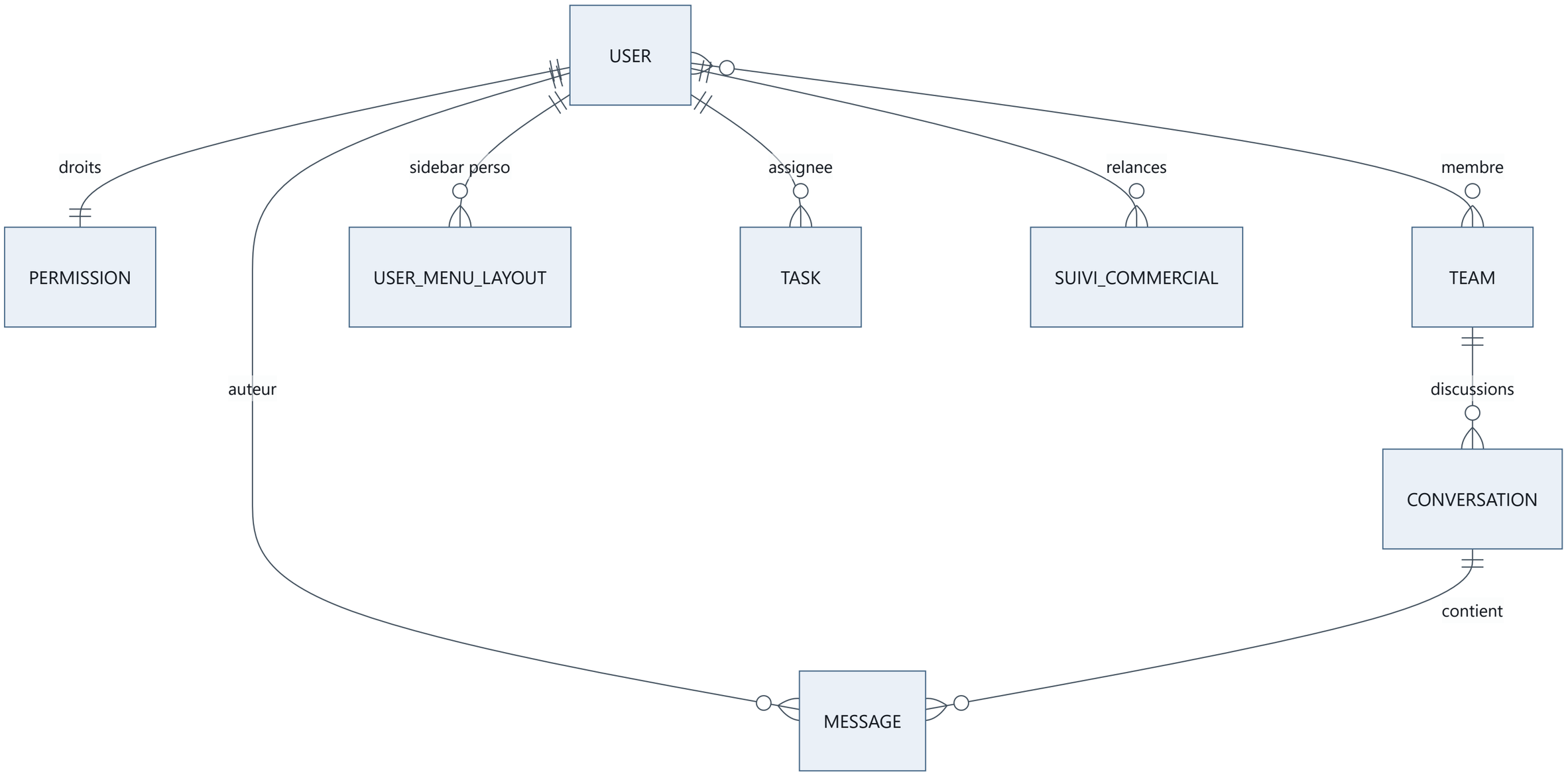
Réappro et bipage ne posent PAS la même question



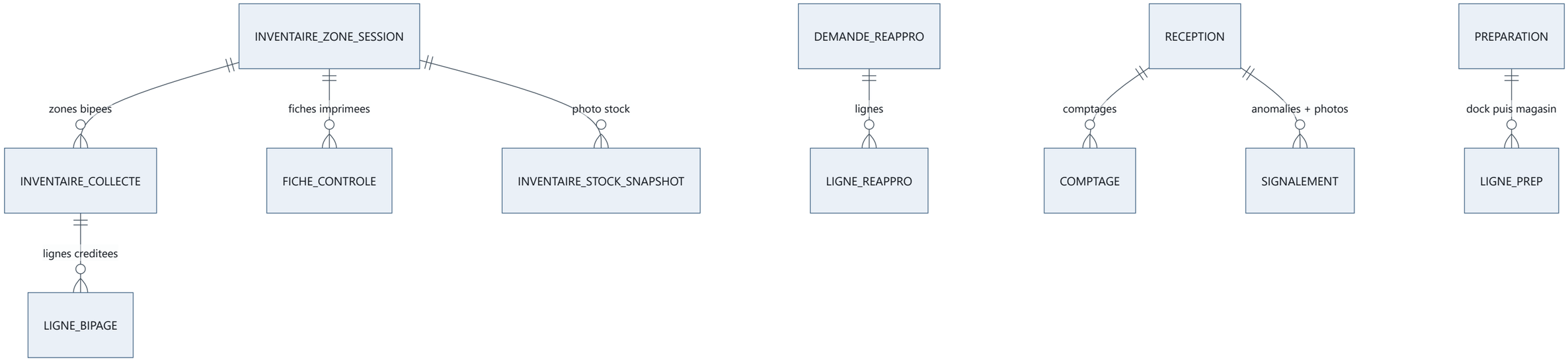
Les entités centrales et leurs liens (67 modèles — seuls les pivots figurent ici).



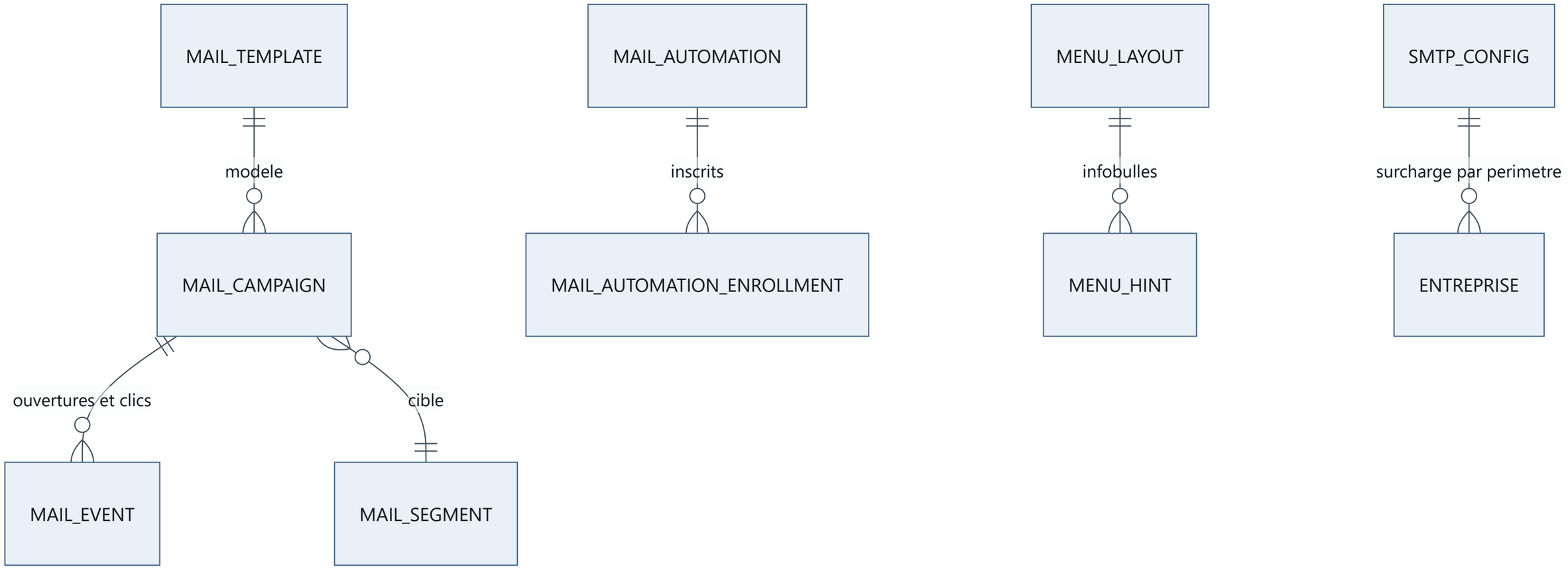
Modèle de données — utilisateurs, équipes et échanges



Modèle de données — les flux du terrain



Modèle de données — mailing et configuration



Les champs qui portent les décisions

Le détail des pivots : ce sont ces champs-là qui décident du comportement.

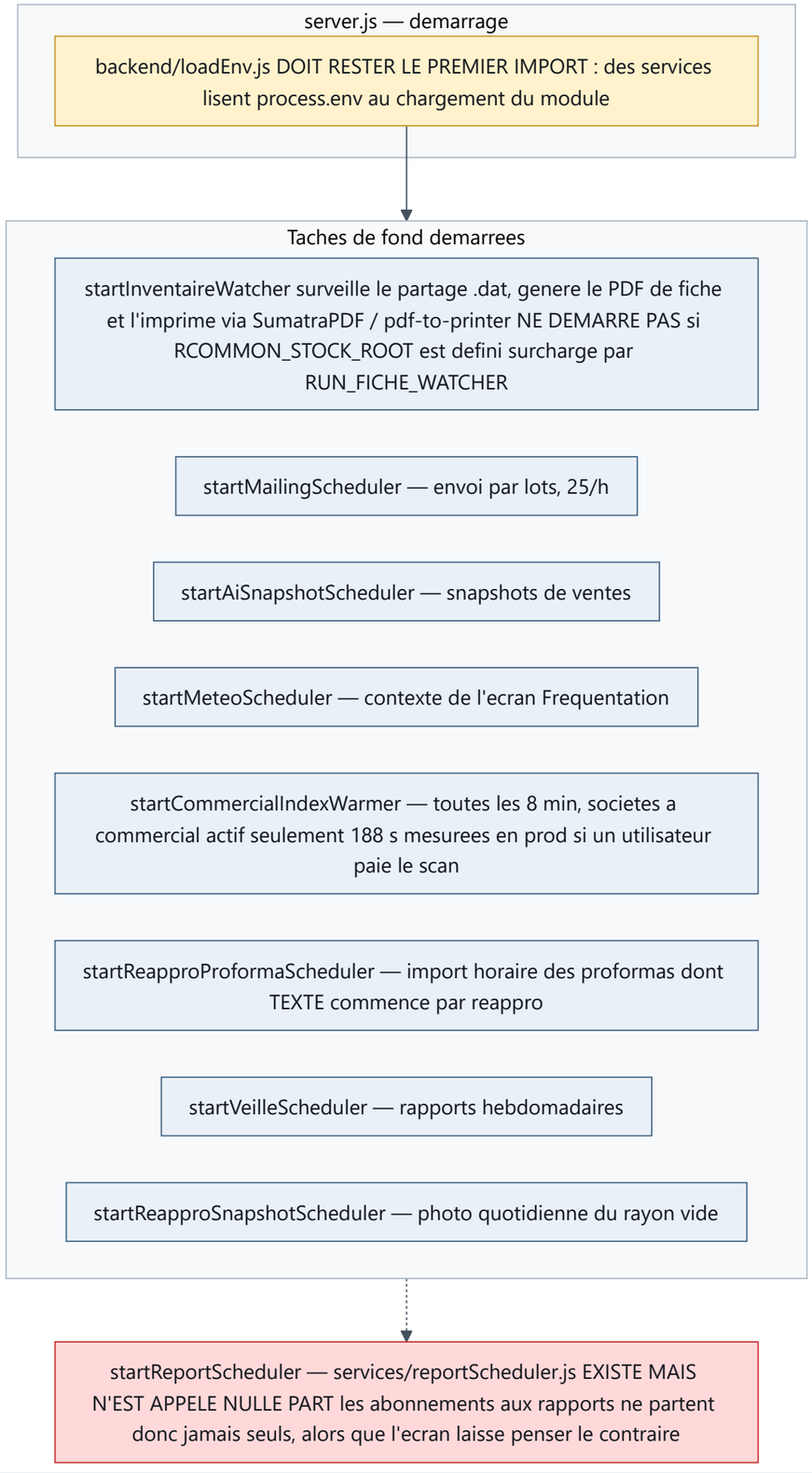
ENTREPRISE		
string	nomDossierDBF	cle de scoping DBF
string	trigramme	QC, KQ, MQ, DQ, HD, SIT
string	cheminBase	surcharge par DBF_BASE_PATH
string	cheminExportInventaire	collect_sec
string	cheminRapportReception	
string	cheminPhotos	
bool	miroirFtpInventaire	defaut true
string	modeDepotDat	rcommun ou ftp
array	vendeurs	code REPRES + type
object	mappingEtatsProforma	
object	mappingEtatsReservation	
object	mappingEntrepots	S2..S5

PERMISSION		
object	modules	cle vers read write delete
array	entreprises	
bool	allEntreprises	super-admin
object	champsDbf	par table ERP
object	commercial	actif + codes par societe

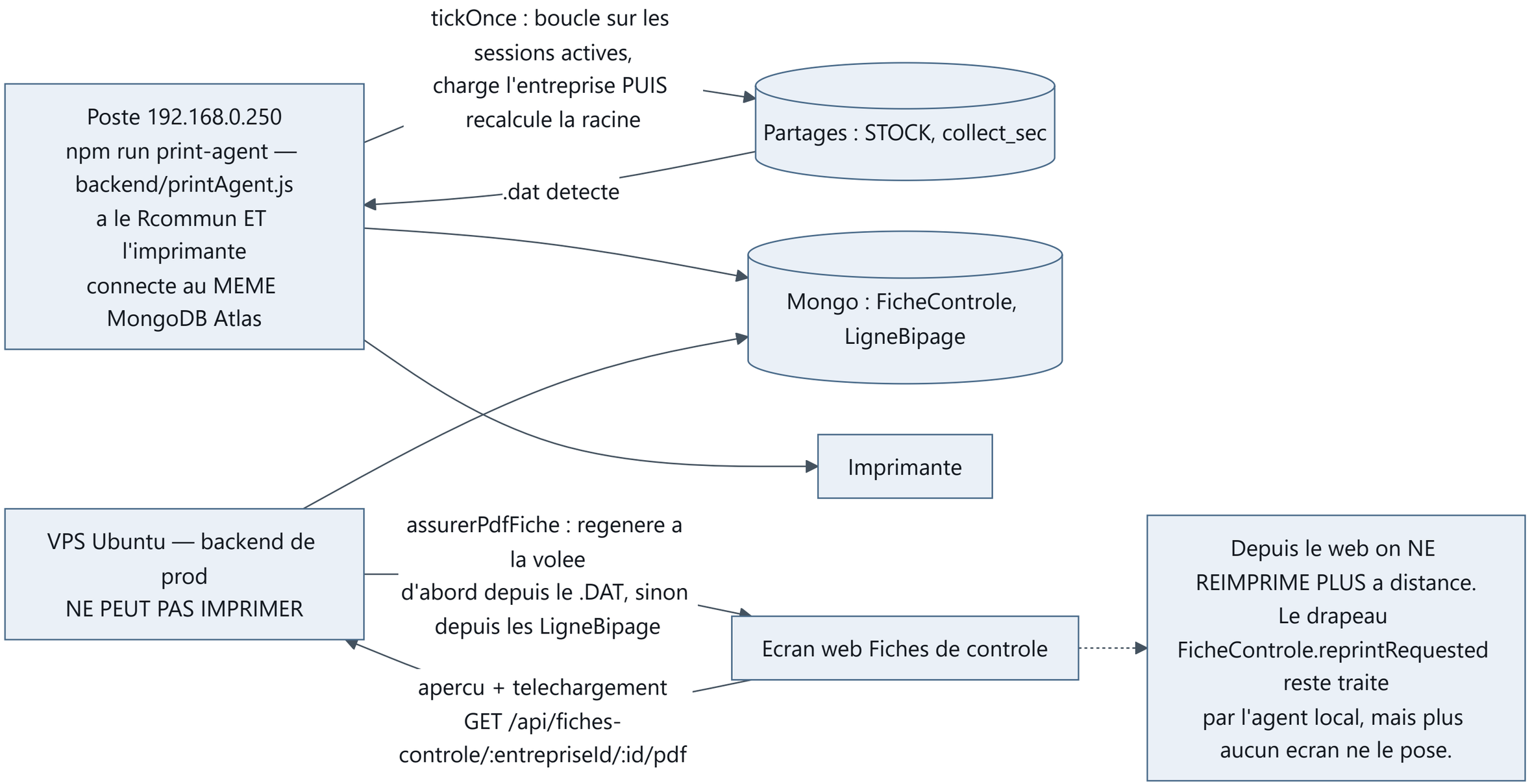
INVENTAIRE_ZONE_SESSION		
string	slug	horodate
string	destination	rcommun ou collect_sec
string	dossierDat	INDICATIF — jamais pour ecrire
string	dossierFtp	miroir de reference
string	statut	

INVENTAIRE_COLLECTE		
string	zoneCode	
string	emplacement	MAGASIN ou DOCK
bool	estComplement	re-bipage = collecte de PLUS
string	statut	en_cours deposee annulee
array	lignes	codeScanne + quantite
string	offlineId	idempotence du depot hors ligne

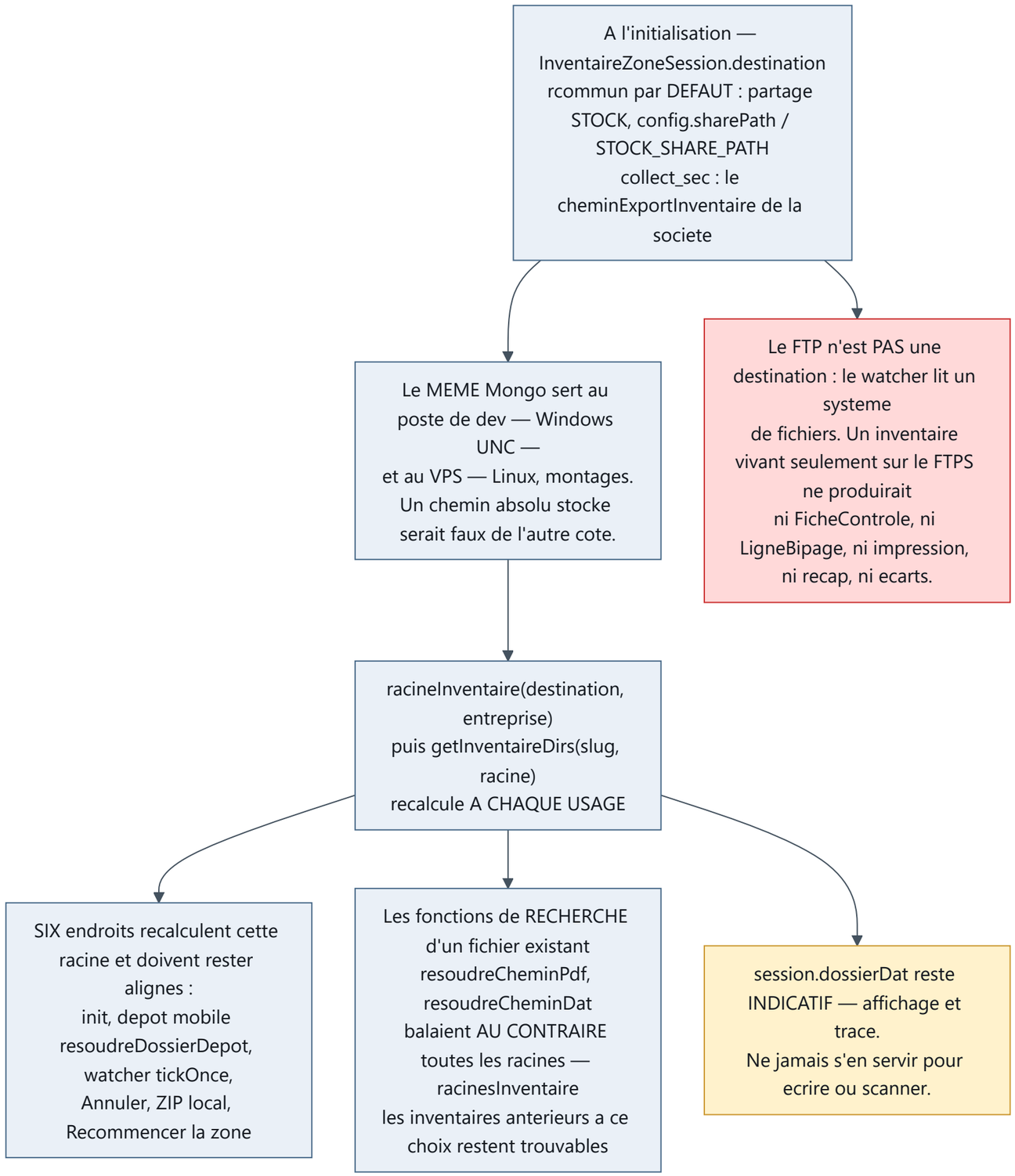
DEMANDE_REAPPRO		
string	source	manuel proforma rapport
string	sourceRef	NUMFACT — index unique partiel
string	statut	en_attente en_cours realisee
object	operateur	VERROU user
string	nommageTransfert	gisement proforma libre


server.js — demarrage
backend/loadEnv.js DOIT RESTER LE PREMIER IMPORT : des services lisent process.env au chargement du module
Taches de fond demarrees
startInventaireWatcher surveille le partage .dat, genere le PDF de fiche et l'imprime via SumatraPDF / pdf-to-printer NE DEMARRE PAS si RCOMMON_STOCK_ROOT est defini surcharge par RUN_FICHE_WATCHER
startMailingScheduler — envoi par lots, 25/h
startAiSnapshotScheduler — snapshots de ventes
startMeteoScheduler — contexte de l'ecran Frequentation
startCommercialIndexWarmer — toutes les 8 min, societes a commercial actif seulement 188 s mesurees en prod si un utilisateur paie le scan
startReapproProformaScheduler — import horaire des proformas dont TEXTE commence par reappro
startVeilleScheduler — rapports hebdomadaires
startReapproSnapshotScheduler — photo quotidienne du rayon vide
startReportScheduler — services/reportScheduler.js EXISTE MAIS N'EST APPELE NULLE PART les abonnements aux rapports ne partent donc jamais seuls, alors que l'ecran laisse penser le contraire

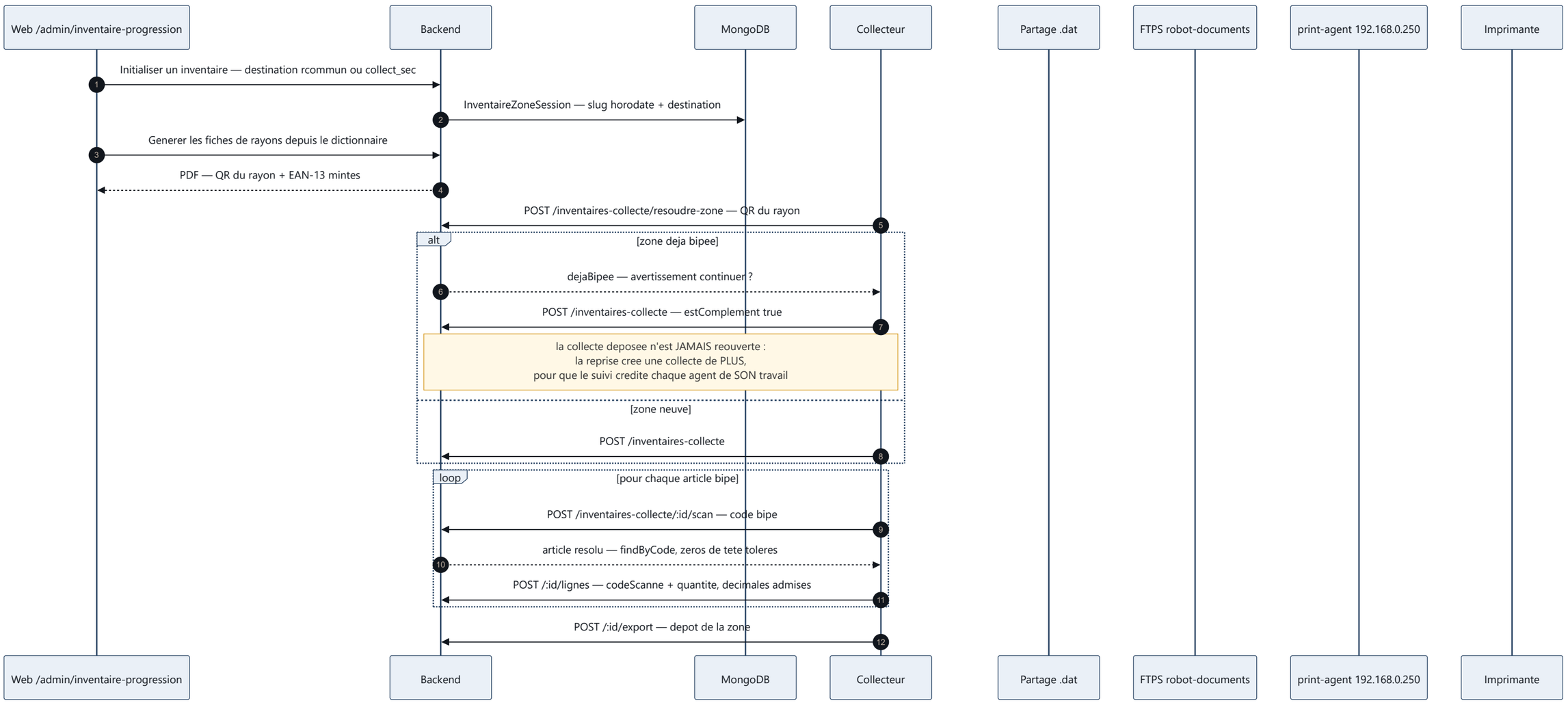
Impression des fiches de contrôle — qui fait quoi



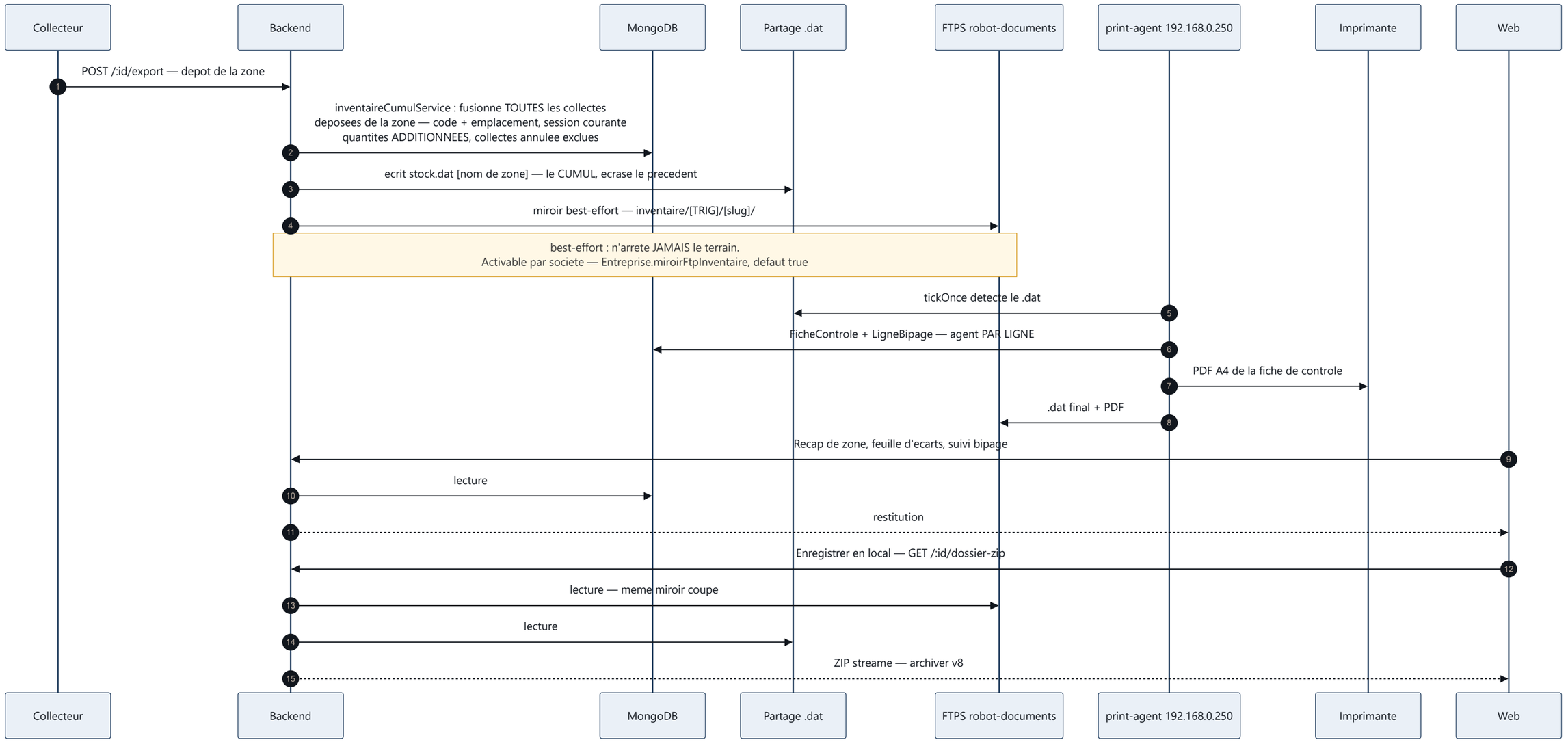
Destination du dossier d'inventaire — on stocke le MODE, jamais un chemin



Le flux qui traverse web, mobile, partage fichier et imprimante.

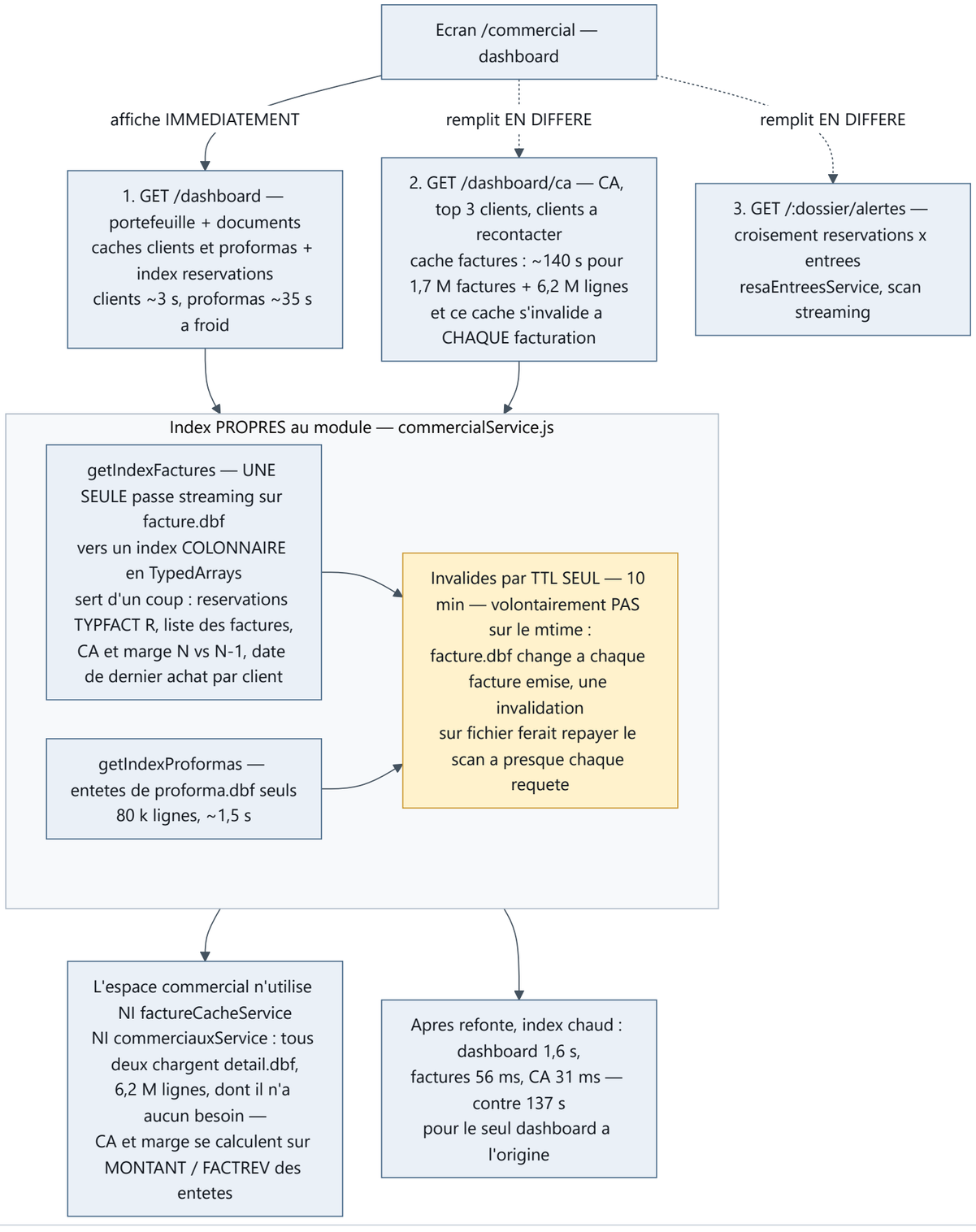


Du dépôt à la fiche imprimée



Re-bipage d'une zone : un second agent rebipe le QR, l'app avertit, et ses bipages **s'ajoutent**. « Recommencer la zone » est au contraire une **table rase** : les collectes passent en `annulee`, exclues du cumul, du récap et des écarts — sans ce drapeau le bipage suivant ferait ressusciter les lignes jetées. Le verrou du **coupon** bipage (re-bipage refusé si la fiche est imprimée) est un autre sujet.

Le dashboard est **découpé en trois requêtes** parce que les caches DBF n'ont pas du tout le même coût à froid sur QC. **Ne pas refusionner ces endpoints.**



Deux sources distinctes, ne pas les confondre

RESERVATIONS et COMMANDES SPECIALES
facture.dbf TYPFACT = R
ETAT 1 = Reservation Stock
ETAT 2 = Commande Speciale
entreprise.mappingEtatsReservation
Meme source que Entrees sur reservation et que les alertes

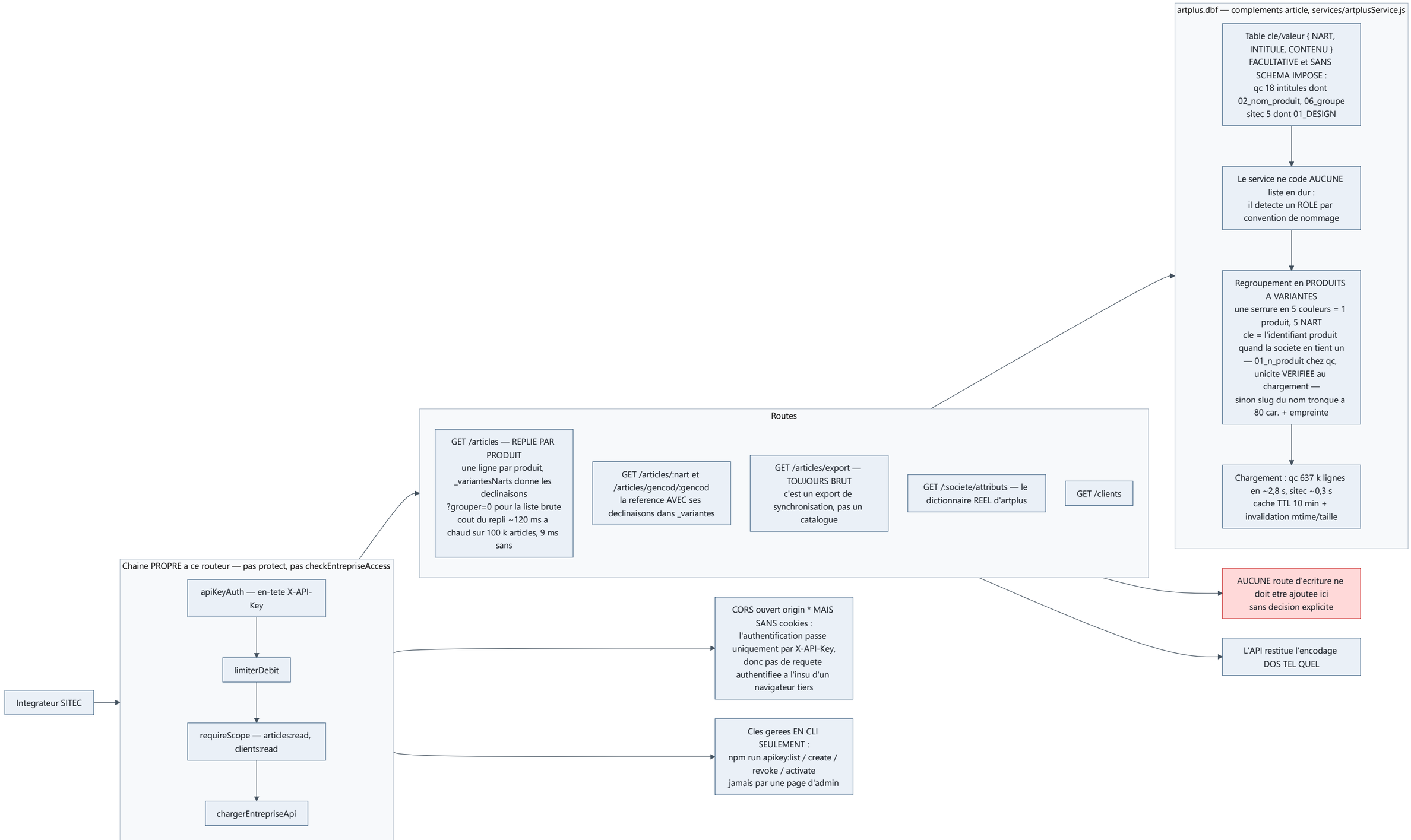
PROFORMAS — proforma.dbf
etats reels chez QC : 1 — 685 doc./12 mois pour un seul commercial,
2 = Commande a preparer, 3 et 4 = devis
ETAT 0 N'EXISTE PAS : l'ancienne convention
0 = commande speciale est FAUSSE

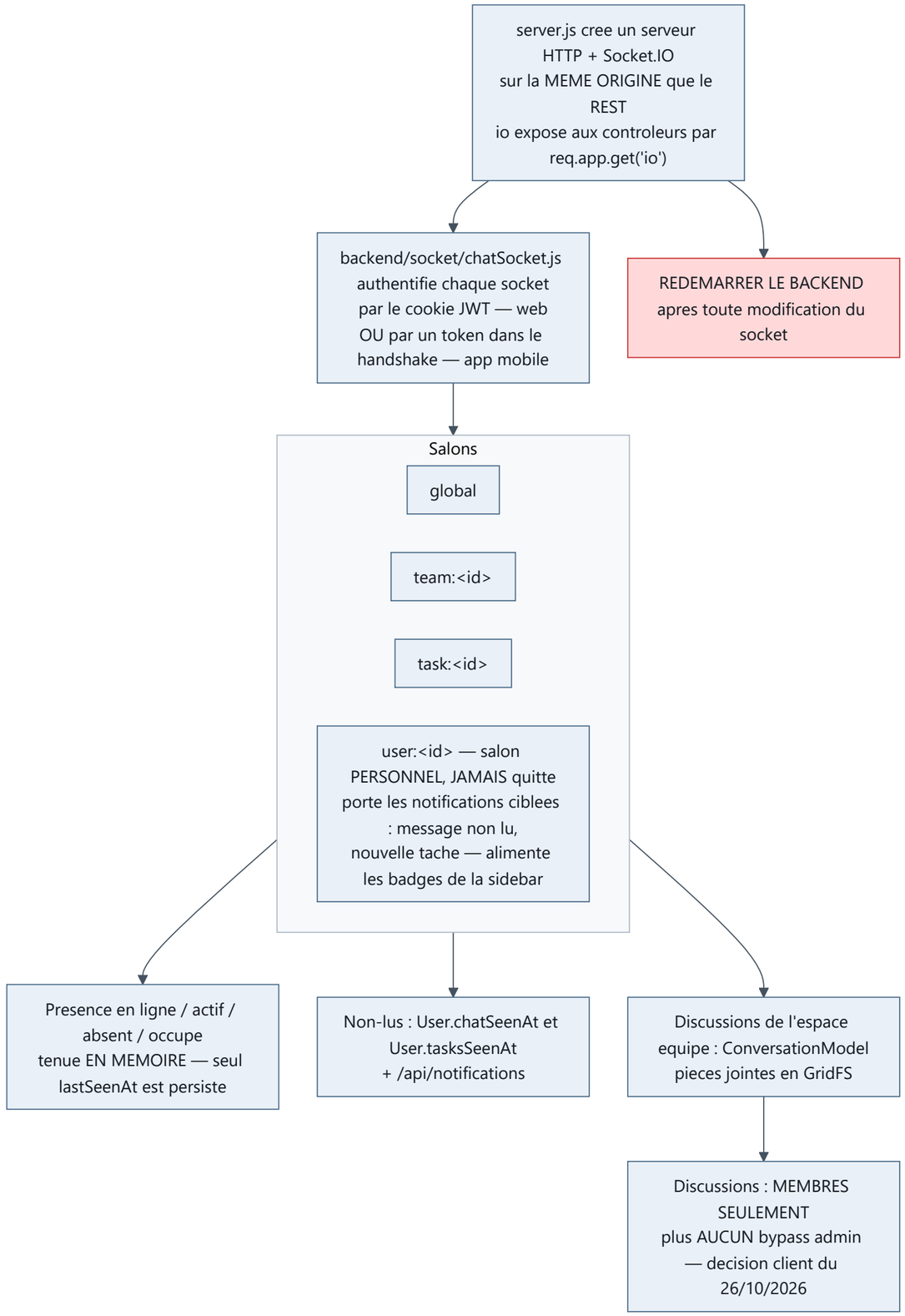
Le mappingEtatsProforma de QC libelle l'ETAT 1 Reservation,
ALORS QUE CE N'EN EST PAS UNE.
On affiche donc le libelle de CATEGORIE — ex. Proforma en attente —
et non le libelle ERP, garde en etatLabelErp pour l'infobulle.
Sans ca, 685 lignes Reservation cotoyaient les 33 vraies.

L'ERP ne purge JAMAIS ces tables : tous les compteurs en cours et a relancer sont bornes par une FENETRE GLISSANTE
de 12 mois — FENETRE_MOIS_DEFAUT, fenetreMois=0 pour tout l'historique.
Sans elle on remonte a 2019 : 4686 reservations en cours au lieu de 33.

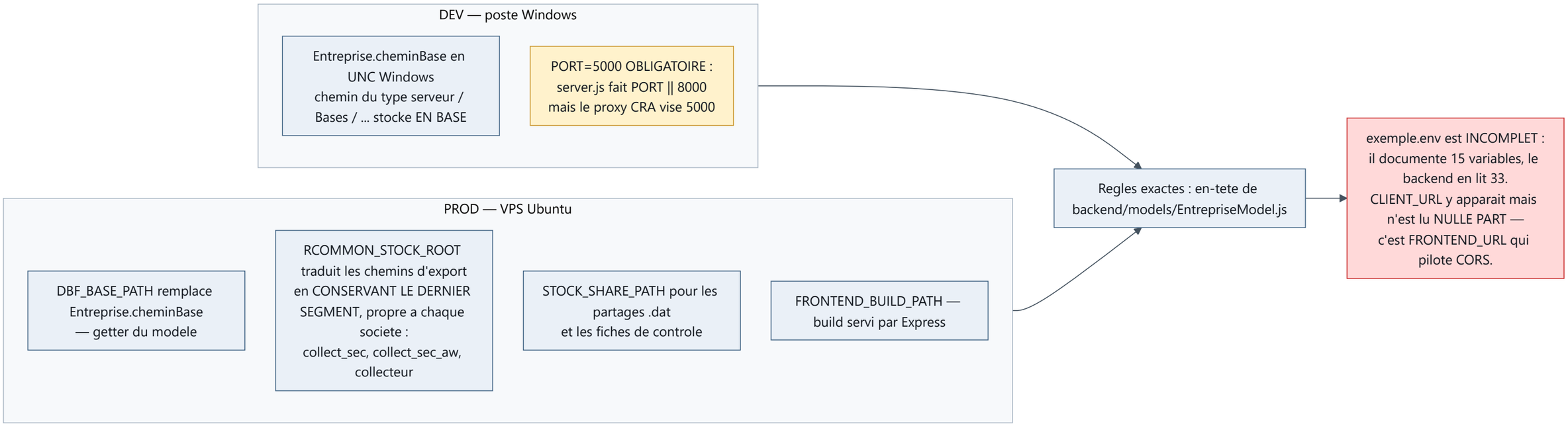
L'écran **Réservations / Commandes spéciales** suit le même découpage : la liste (entêtes, rapide), puis `/reservations/disponibilites` (croisement `detail.dbf × entrees.dbf`, **142 s à froid en local**), puis le détail ligne à ligne à l'ouverture. Une ligne est « arrivée » quand l'article a une entrée **postérieure ou égale** à la date de la réservation ; au-delà de la profondeur du scan (`FENETRE_DISPO_MOIS` = 24 mois) le statut est `inconnu`, pas « en attente ».

Accès externe **en lecture seule** pour un intégrateur (site marchand SITEC). Contrat destiné au prestataire : `docs/API_PARTENAIRE.md` — à mettre à jour **en même temps que le code**.




server.js cree un serveur HTTP + Socket.IO sur la MEME ORIGINE que le REST
io expose aux controleurs par req.app.get('io')
backend/socket/chatSocket.js
authentifie chaque socket par le cookie JWT — web OU par un token dans le handshake — app mobile
REDEMARRER LE BACKEND apres toute modification du socket
Salons
global
team:<id>
task:<id>
user:<id> — salon PERSONNEL, JAMAIS quitte
porte les notifications ciblees : message non lu, nouvelle tache — alimente les badges de la sidebar
Presence en ligne / actif / absent / occupe
tenue EN MEMOIRE — seul lastSeenAt est persiste
Non-lus : User.chatSeenAt et User.tasksSeenAt
+ /api/notifications
Discussions de l'espace equipe : ConversationModel
pieces jointes en GridFS
Discussions : MEMBRES SEULEMENT
plus AUCUN bypass admin — decision client du 26/10/2026

Les chemins des sociétés sont stockés en **UNC Windows** (`\\serveur\Bases\…`) pour le développement local. En production (Ubuntu) des variables d'environnement les remplacent ou les traduisent **sans aucune migration en base**.



Domaine	Variables
Base	`NODE_ENV` , `PORT` , `MONGO_URI` (alias `MONGODB_URI`), `JWT_SECRET`
Front / CORS	`FRONTEND_URL` , `FRONTEND_BUILD_PATH`
Chemins DBF & partages	`DBF_BASE_PATH` , `RCOMMON_STOCK_ROOT` , `RCOMMON_COLLECT_PATH` , `STOCK_SHARE_PATH` , `PHOTOS_BASE_PATH` , `REAPRO_MAG_DIR` , `ANALYSE_CA_DICTIONNAIRE_PATH`
Impression fiches	`RUN_FICHE_WATCHER` , `PRINTER_NAME` , `FICHE_AUTOPRINT` , `FICHE_WATCH_INTERVAL_MS` , `DAT_FILENAME_PREFIX` , `DAT_FILENAME_REGEX`
Email	`SMTP_HOST` , `SMTP_PORT` , `SMTP_SECURE` , `SMTP_USER` , `SMTP_PASSWORD` , `SMTP_FROM_NAME` , `MAIL_TRACK_SECRET`
Envoi Cde Fournisseur	`ENVOI_CDE_TEST_EMAILS` (le mode test se pilote **par société dans l'UI**)
IA	`OPENAI_API_KEY` , `OPENAI_MODEL` , `TAVILY_API_KEY` , `OPENAI_VEILLE_MODEL` , `OPENAI_VEILLE_MODEL_PRO`
Frontend (CRA)	`REACT_APP_MAPBOX_TOKEN` , `REACT_APP_API_TARGET`

Commandes

npm run dev — backend nodemon + frontend CRA, concurrently	npm run server — backend seul	npm run client — front seul, port 3000, proxy /api vers 5000	npm start — prod, node backend/server.js	npm run print-agent — agent d'impression local
data:import / data:destroy — users, permissions, entreprises data:import:envoi-cde — module Envoi Cde masterconfig:import — configs master report	apikey:list / create / revoke / activate — CLI UNIQUEMENT	pachat:historiser — historisation des prix d'achat migrate:groupes-prioritaires — migration ponctuelle	cd frontend puis npm run build VIDER NODE_OPTIONS et CI avant, sinon dns-fix.js echoue	npm test existe mais NE TROUVE AUCUN TEST ne pas s'en servir comme preuve

Détail et priorisation dans `docs/AUDIT_APPLICATION.md`.

Etats des documents

Les pastilles proforma / facture / commande / reservation viennent des mappingEtats* de la fiche societe — JAMAIS de libelles en dur. mappingEtatsFacture est vide partout.

Garde-fous d'envoi — a ne jamais relacher

Les tests de mailing ne doivent atteindre QUE communication@ / support@quincaillerie.nc et krysto.contact@gmail.com — JAMAIS la base clients

Le module Envoi Cde Fournisseur est en MODE TEST PAR DEFAUT, pilote par societe dans l'UI

Absence de filet

AUCUN TEST. Les fonctions pures a fort enjeu n'ont aucun filet : calcul TGC, format .dat, assertGrantWithinScope, perimetre commercial

PAS de limitation de debit sur /api/users/login : limiterDebit n'existe que pour l'API partenaire

exemple.env ne suffit pas a monter une instance — audit 3.5

Code mort — audit 3.4

AdminProformaScreen.jsx

AdminDoublonsGencodeScreen.jsx

frontend/src/utils/api.js — utilise NEXT_PUBLIC_*, jamais expose par CRA

/api/dashboard/* sans consommateur web

Derives structurelles

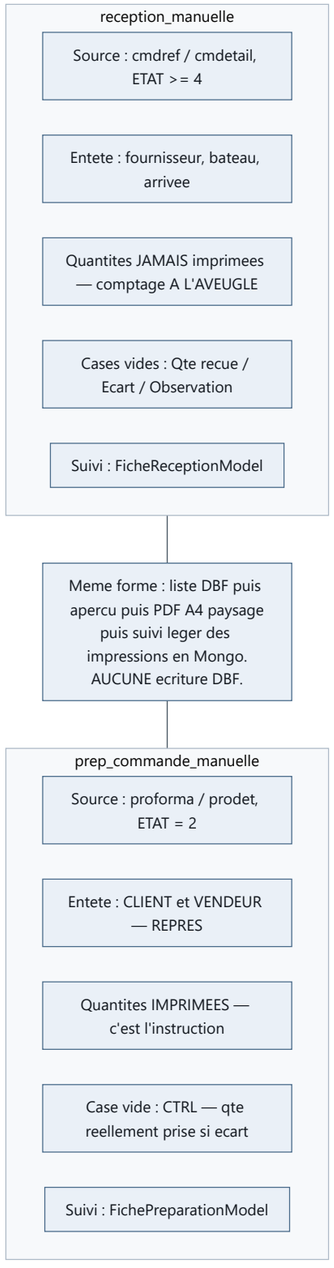
Registres de permission DESYNCHRONISES 3 sources, 6 cles divergentes — audit 3.1

backend/routes/InventaireRoutes.js ne contient PLUS les routes d'inventaire : depuis le commit 7c5bf4f c'est un CLONE des routes de zones. /api/inventaires et /api/inventaires-zones servent les memes routes avec DEUX politiques d'acces differentes, et inventaireController.js — 1052 lignes — est ORPHELIN. Audit 3.2

DEUX routeurs montes sur /api/filiales : fillialeRoutes = comparatif par article, filialesRoutes = analyse reseau DQ/QC/LD. Ca ne marche que parce que leurs formes d'URL ne se recouvrent pas. Audit 3.3

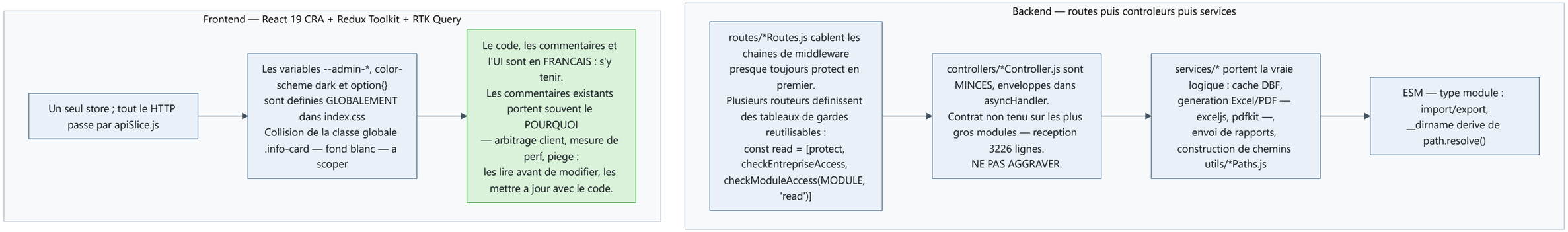
startReportScheduler jamais appele : les abonnements aux rapports ne partent jamais seuls

Deux modules jumeaux, deux règles métier OPPOSÉES



Parcours de la fiche de préparation : le PDF est découpé en **1 · DOCK** puis **2 · MAGASIN**, et l'on commence *toujours* par le dock — `qteDock = min(qté demandée, S2)` , le reste part au magasin (S1). Un article peut donc apparaître **deux fois**. La répartition n'est **pas redéveloppée** : `preparationManuelleService` appelle `analyserProforma()` de `services/preparationService.js` , celui de la préparation scannée.

Colonnes, dans l'ordre : case à cocher, `NL` , `CODE` , `DÉSIGNATION` , `GENCODE` , `FOURNISSEUR` , `RÉF.` , `GISEMENT` , `RAYON` , puis le trio `QTÉ CDE` / `À PRENDRE` / `CTRL` . **Ce trio ne change jamais d'ordre** — retirer `QTÉ CDE` rend la fiche illisible dès qu'un article est à cheval. La colonne de repères et la colonne `DISPO` ont été **retirées le 10/09/2026** à la demande du client : ne pas les réintroduire sans décision explicite.


Frontend — React 19 CRA + Redux Toolkit + RTK Query
Un seul store ; tout le HTTP passe par apiSlice.js
Les variables --admin-*, color-scheme dark et option{} sont definies GLOBALEMENT dans index.css
Collision de la classe globale .info-card — fond blanc — a scoper
Le code, les commentaires et l'UI sont en FRANCAIS : s'y tenir.
Les commentaires existants portent souvent le POURQUOI
— arbitrage client, mesure de perf, piege :
les lire avant de modifier, les mettre a jour avec le code.
Backend — routes puis controleurs puis services
routes/*Routes.js cablent les chaines de middleware presque toujours protect en premier.
Plusieurs routeurs definissent des tableaux de gardes reutilisables :
const read = [protect, checkEntrepriseAccess, checkModuleAccess(MODULE, 'read')]
controllers/*Controller.js sont MINCES, enveloppes dans asyncHandler.
Contrat non tenu sur les plus gros modules — reception 3226 lignes.
NE PAS AGGRAVER.
services/* portent la vraie logique : cache DBF, generation Excel/PDF — exceljs, pdfkit —, envoi de rapports, construction de chemins utils/*Paths.js
ESM — type module : import/export, __dirname derive de path.resolve()